

UNIVERSIDADE CESUMAR – UNICESUMAR

AMANDA RODRIGUES NEVES

JULIA OLIVEIRA DOS SANTOS

JULIA YUMI HASHIMOTO

REHABITA

Sistema de gestão de doações e acompanhamento de famílias em situação de vulnerabilidade social

Atividade de Estudo Programada (AEP) – 4º semestre

1ª Entrega: Planejamento, Escopo e Estruturação (1º bimestre)

MARINGÁ – PR

2026

1. Descoberta

1.1 Como surgiu a ideia

A ideia do REHABITA surgiu observando o trabalho de instituições de Maringá que ajudam famílias em situação de vulnerabilidade, como paróquias, igrejas, pastorais e projetos sociais de bairro. Essas instituições recebem doações o tempo todo (cesta básica, produtos de higiene, fraldas, roupas, cobertores) e repassam para famílias da comunidade que estão passando necessidade.

O que chamou a nossa atenção foi a forma como tudo isso é controlado. Na maioria dos casos o registro fica em caderno, em uma planilha feita por algum voluntário ou em grupo de WhatsApp. Funciona enquanto são poucas famílias e as mesmas pessoas cuidam de tudo, mas basta trocar o voluntário ou aumentar a procura para a informação começar a se perder.

Nas conversas com pessoas que atuam nesse tipo de trabalho, algumas perguntas apareciam sempre e ninguém conseguia responder rápido: quantas cestas ainda temos guardadas? Essa família já recebeu este mês? Quem fez a última visita e o que ficou combinado? Quais famílias estão em situação mais grave agora? Foi a partir dessas perguntas que montamos o levantamento de dores e, depois, a lista de requisitos.

A proposta, então, é um sistema para a própria instituição organizar o ciclo completo: a doação que chega, o estoque, a família que é cadastrada e visitada e o que foi entregue para ela. Quem usa o sistema é a equipe da instituição, ou seja, a coordenação e as pessoas responsáveis pelas visitas.

1.2 Partes interessadas

Identificamos quatro grupos envolvidos. Só os dois primeiros usam o sistema diretamente. Os outros dois não acessam a aplicação, mas são afetados pelo resultado dela.

Coordenação da instituição: é quem cuida da arrecadação e da distribuição nas paróquias, igrejas e projetos sociais.

Assistentes sociais, agentes de pastoral e voluntários de visita: fazem o cadastro das famílias e as visitas.

Famílias assistidas: são as mais afetadas pelo resultado. Com o cadastro organizado, a família não precisa repetir a mesma história a cada visita e tem menos chance de ficar esquecida quando a situação piora.

Doadores: pessoas, empresas e campanhas de arrecadação (de escolas, rádios, comércios) que entregam as doações. Eles não acessam o REHABITA, quem registra a doação é a própria instituição.

1.3 Dores identificadas e origem dos problemas

O quadro abaixo resume os problemas que mais apareceram, por que eles acontecem e qual parte do sistema responde a cada um. Os códigos R e RN são os requisitos e as regras de negócio.

Quadro 1 – Dores, origens e resposta do sistema

O problema (dor)	Por que acontece	O que o REHABITA faz
A informação da família se perde ou precisa ser perguntada de novo a cada visita.	Os dados ficam em cadernos, planilhas e conversas de WhatsApp, e cada voluntário tem a sua versão.	Cadastro único da família e dos membros (R1, R2).
Algumas famílias recebem duas vezes no mesmo mês e outras ficam sem nada.	Não existe um lugar onde dê para ver o que cada família já recebeu e quando.	Histórico de entregas e aviso de entrega repetida (R6, RN4).
Casos urgentes esperam na mesma fila que os outros.	Não há um jeito combinado de marcar urgência nem de lembrar quem precisa de retorno.	Nível de urgência, data de retorno e lista de prioridade (R3, R9, RN7).
A coordenação não sabe quanto tem em estoque nem quando algo vai acabar.	As doações chegam por vários lados e são guardadas sem anotar entrada e saída.	Registro das doações recebidas e saldo por item (R4, R5, RN3).
É difícil prestar contas para quem doou e para a própria instituição.	Nada liga o que entrou com o que foi entregue.	Doações e entregas ficam gravadas com data e responsável (R5, R6).
Não dá para saber quais bairros concentram mais famílias atendidas.	O endereço é anotado de qualquer jeito, então não dá para agrupar.	Bairro escolhido de uma lista e consulta por bairro (R1, R9).

2. Concepção do projeto e alinhamento às ODS

2.1 O que é o REHABITA

O REHABITA é um sistema de aplicação web para instituições que distribuem doações a famílias em situação de vulnerabilidade social. Ele junta em um lugar só o cadastro das famílias, o registro das visitas, o controle das doações recebidas e o histórico do que foi entregue para cada família.

2.2 Como o sistema funciona no dia a dia

O sistema segue o mesmo caminho que a doação já faz hoje dentro da instituição, só que agora tudo fica registrado:

1. Chega uma doação. A coordenação lança o que entrou (itens e quantidades) e quem doou, se o doador quiser se identificar. As quantidades somam no estoque.
2. Uma família é indicada ou procura a instituição. É feito o cadastro do responsável, do endereço e dos membros, e a família entra com status Novo.

www.unicesumar.edu.br

3. Alguém da equipe faz a visita e registra o atendimento, marcando o nível de urgência e quando deve voltar. A família passa para Em acompanhamento.
4. Se houver entrega nesse atendimento, os itens são lançados ali mesmo. O sistema confere o saldo, avisa se a família já recebeu o mesmo item há pouco tempo e desconta do estoque.
5. A coordenação consulta a lista de prioridade, o saldo dos itens e as famílias por bairro para decidir as próximas visitas e quando fazer uma nova campanha.

2.3 ODS atendidas

O projeto se relaciona principalmente com três Objetivos de Desenvolvimento Sustentável da ONU. Ligamos cada ODS a uma meta específica e às funcionalidades que atendem essa meta.

ODS 1 – Erradicação da Pobreza: A meta trata de proteção social que realmente alcance as pessoas pobres e vulneráveis. As instituições de bairro fazem parte dessa rede e muitas vezes são o primeiro lugar que a família procura. O REHABITA contribui porque cada família cadastrada tem status, histórico de visitas e data de retorno (R3, R7), então fica visível quando alguém está há muito tempo sem atendimento.

ODS 2 – Fome Zero e Agricultura Sustentável: A maior parte do que essas instituições distribuem é alimento, principalmente a cesta básica, e essa meta fala justamente em garantir acesso a alimento para quem está em situação de vulnerabilidade. Com o controle de estoque e de entregas (R4, R5 e R6), a coordenação percebe que a cesta está acabando antes de faltar e consegue organizar uma campanha a tempo.

ODS 10 – Redução das Desigualdades. A meta fala em inclusão social de todas as pessoas. No nosso caso isso aparece de um jeito bem prático: a lista de prioridade e o aviso de entrega repetida (RN4 e RN7) fazem a ajuda ser distribuída por critério, ou seja, pela urgência e pelo tempo sem receber.

3. Escopo: lista de requisitos

3.1 Requisitos

Os requisitos abaixo definem o que o sistema vai fazer na 2ª entrega, e cada um veio de uma dor da seção 1.3. A entidade principal, sobre a qual será feito o CRUD completo, é a Família (R1).

Quadro 2 – Lista de requisitos

Nº	Requisito
R1	O sistema deve permitir o cadastro de famílias assistidas, com os dados do responsável (nome completo, CPF, data de nascimento, telefone, escolaridade e situação de trabalho), o endereço (CEP, rua, número, complemento e bairro), o tipo de moradia e a faixa de renda, além de consultar, editar e excluir esse cadastro.
R2	O sistema deve permitir o cadastro dos membros de cada família, com nome completo, data de nascimento, grau de parentesco com o responsável, telefone e CPF (esses dois opcionais), calculando automaticamente quantas pessoas moram na casa.

Nº	Requisito
R3	O sistema deve permitir o registro dos atendimentos e visitas feitos a cada família, com data da visita, nível de urgência (baixa, média ou alta), usuário que atendeu, observações e data prevista para o retorno.
R4	O sistema deve permitir o cadastro dos itens distribuídos pela instituição (ex.: cesta básica, kit de higiene, fralda, cobertor), com nome, categoria (alimento, higiene, vestuário ou outros) e unidade de medida, mostrando o saldo atual em estoque de cada item.
R5	O sistema deve permitir o registro das doações recebidas, com data, doador (pessoa física, empresa, instituição ou doação anônima) e os itens com suas quantidades, somando essas quantidades ao estoque.
R6	O sistema deve permitir o registro dos itens entregues à família em cada atendimento, com a quantidade, descontando do estoque e mantendo o histórico de tudo o que a família já recebeu.
R7	O sistema deve permitir a alteração do status de acompanhamento da família (Novo, Em acompanhamento ou Inativo), guardando a data, o usuário e o motivo de cada mudança.
R8	O sistema deve permitir o cadastro de usuários (nome, CPF, telefone, e-mail, senha e perfil) e o login com e-mail e senha, com dois perfis: coordenador, que acessa tudo e é o único que cadastra usuários, itens e bairros, e assistente social, que trabalha com famílias, atendimentos, entregas, doações e consultas.
R9	O sistema deve permitir consultar as famílias por bairro, status e nível de urgência, gerar a lista de prioridade de atendimento e o relatório de saldo dos itens em estoque.

3.2 Regras de negócio

Durante o levantamento apareceram algumas regras que o sistema precisa respeitar. No código elas ficam na camada service e, quando dá, também no banco de dados, como uma segunda proteção.

Quadro 3 – Regras de negócio

Código	Regra	Requisito
RN1	Não pode existir duas famílias com o mesmo CPF de responsável. Se o CPF já estiver cadastrado, o sistema mostra a família que já existe em vez de criar outra.	R1
RN2	A quantidade de pessoas da família não é digitada: é o responsável mais os membros cadastrados.	R2
RN3	O saldo de um item só muda por doação recebida (entrada) ou por entrega (saída), nunca é editado à mão. A entrega só é gravada se houver saldo suficiente, e o registro da entrega e a baixa no estoque são salvos juntos: se um falhar, nenhum dos dois é salvo.	R4, R5, R6
RN4	Se a família já recebeu o mesmo item nos últimos 30 dias (a cesta básica costuma ser mensal), o sistema avisa. A entrega só continua se o atendimento estiver marcado com urgência alta.	R6
RN5	Toda família começa com status Novo e passa para Em acompanhamento no primeiro atendimento. Qualquer mudança manual de status exige um motivo e fica gravada no histórico.	R3, R7

Código	Regra	Requisito
RN6	Família que já tem atendimento registrado não pode ser excluída, só inativada, para não apagar o histórico de entregas.	R1, R7
RN7	A lista de prioridade mostra as famílias que não estão inativas, ordenadas pela urgência do último atendimento e, em caso de empate, por quem está há mais tempo sem receber entrega. Família com status Novo entra como urgência média até receber a primeira visita.	R9

4. Cronograma de execução

O cronograma abaixo mostra o que cada integrante vai fazer no 2º bimestre, semana a semana. Cada uma tem tarefas em todas as fases, para que o histórico do GitHub mostre a participação de todas. A mesma tabela está no README do repositório.

Quadro 4 – Cronograma e responsáveis

Semana	Período	Atividade	Requisito	Responsável
1	14/09 a 20/09	Criar o projeto Spring Boot e as pastas do código	–	Julia O.
		Instalar o SQL Server e rodar os scripts do banco	–	Amanda
2	21/09 a 27/09	Escrever as classes de pessoas: Pessoa, Responsavel, Membro e Usuario	R1, R2, R8	Julia Y.
		Fazer a conexão do sistema com o banco funcionar	–	Amanda
3	28/09 a 04/10	Cadastro de famílias: gravar, buscar, listar, editar e excluir	R1	Amanda
		Tela de login e cadastro de usuários	R8	Julia O.
4	05/10 a 11/10	Páginas de lista e cadastro de família	R1	Julia Y.
		Cadastro dos membros da família	R2	Julia O.
5	12/10 a 18/10	Registro das visitas feitas à família	R3	Julia Y.
		Mudança de status da família, com histórico	R7	Amanda
6	19/10 a 25/10	Cadastro dos itens e controle do saldo em estoque	R4	Amanda
		Registro das doações recebidas, somando no estoque	R5	Julia O.
7	26/10 a 01/11	Entrega de itens na visita, descontando do estoque	R6	Julia Y.
		Testar as regras de estoque e de entrega repetida	R6	Amanda
8	02/11 a 08/11	Consultas por bairro, status e urgência, e lista de prioridade	R9	Julia O.
		Relatório de estoque e menu de acordo com o perfil	R8, R9	Julia Y.
9	09/11 a 15/11	Testar o sistema inteiro e corrigir os erros encontrados	Todos	Amanda, Julia O. e Julia Y.

Semana	Período	Atividade	Requisito	Responsável
		Rodar o projeto do zero em outro computador	-	Julia Y.
10	16/11 a 22/11	Escrever no README o passo a passo para rodar o sistema	-	Julia O.
		Revisão final do código e publicação da versão no GitHub	-	Amanda

Obs.: Julia O. = Julia Oliveira dos Santos; Julia Y. = Julia Yumi Hashimoto.

5. Justificativa técnica e arquitetural

O REHABITA vai ter uma interface web, mas o coração do sistema é o back-end escrito em Java: são as classes, as regras de negócio e o acesso ao banco de dados SQL Server. Durante o desenvolvimento, vamos testar essas classes pelo terminal, antes de ligar nas páginas. O código fica separado em camadas. Abaixo explicamos o porquê de cada escolha.

5.1 Linguagem: Java

Escolhemos Java por dois motivos:

Evita repetir código. Responsável, membro e usuário são pessoas e guardam quase os mesmos dados. Entrada e saída de estoque também se parecem: as duas mudam o saldo do item, só que uma soma e a outra desconta. Em Java dá para escrever essa parte comum uma vez só e reaproveitar.

É a linguagem que estamos aprendendo. Java é a linguagem das aulas do semestre. Vamos usar a versão 21, que tem suporte de longo prazo.

5.2 Framework: Spring Boot

Como o sistema vai ser web, escolhemos o Spring Boot, que é o framework Java mais usado para isso. Ele resolve três coisas para a gente:

Não precisa instalar servidor. O Spring Boot já vem com o servidor embutido. O professor roda um comando só e abre o sistema no navegador, sem configurar mais nada. Isso reduz bastante o risco de o projeto não rodar na máquina dele.

Facilita a conexão com o banco. Os dados de acesso ao SQL Server ficam num arquivo de configuração separado do código. Para trocar de computador, muda só esse arquivo.

As páginas são feitas em HTML. Usamos o Thymeleaf, que deixa montar as telas em HTML comum, puxando os dados que vêm do Java.

5.3 Banco de dados: SQL Server

Os dados do REHABITA são bem ligados uns aos outros: a família tem membros, visitas e histórico; cada visita pode ter itens entregues; cada doação tem itens recebidos. Por isso escolhemos um banco relacional, que garante essas ligações sozinho.

Escolhemos o SQL Server por dois motivos. É o banco que estamos praticando na aula de Banco de Dados, e ele protege sozinho quatro regras do nosso sistema:

Só aceita os valores certos. O banco recusa um status ou uma urgência fora da lista e não deixa o saldo do estoque ficar negativo (RN3). Mesmo se escapar um erro no código, o dado errado não entra.

Aceita CPF em branco, mas não repetido. Criança muitas vezes não tem CPF, então o campo é opcional. Mas, quando é preenchido, o banco não deixa cadastrar o mesmo CPF duas vezes.

Cuida do que apagar junto. Se a família for excluída, os membros e o histórico dela somem junto, porque não existem sozinhos. Já a família que tem visita registrada o banco não deixa excluir (RN6), para não perder o histórico de entregas.

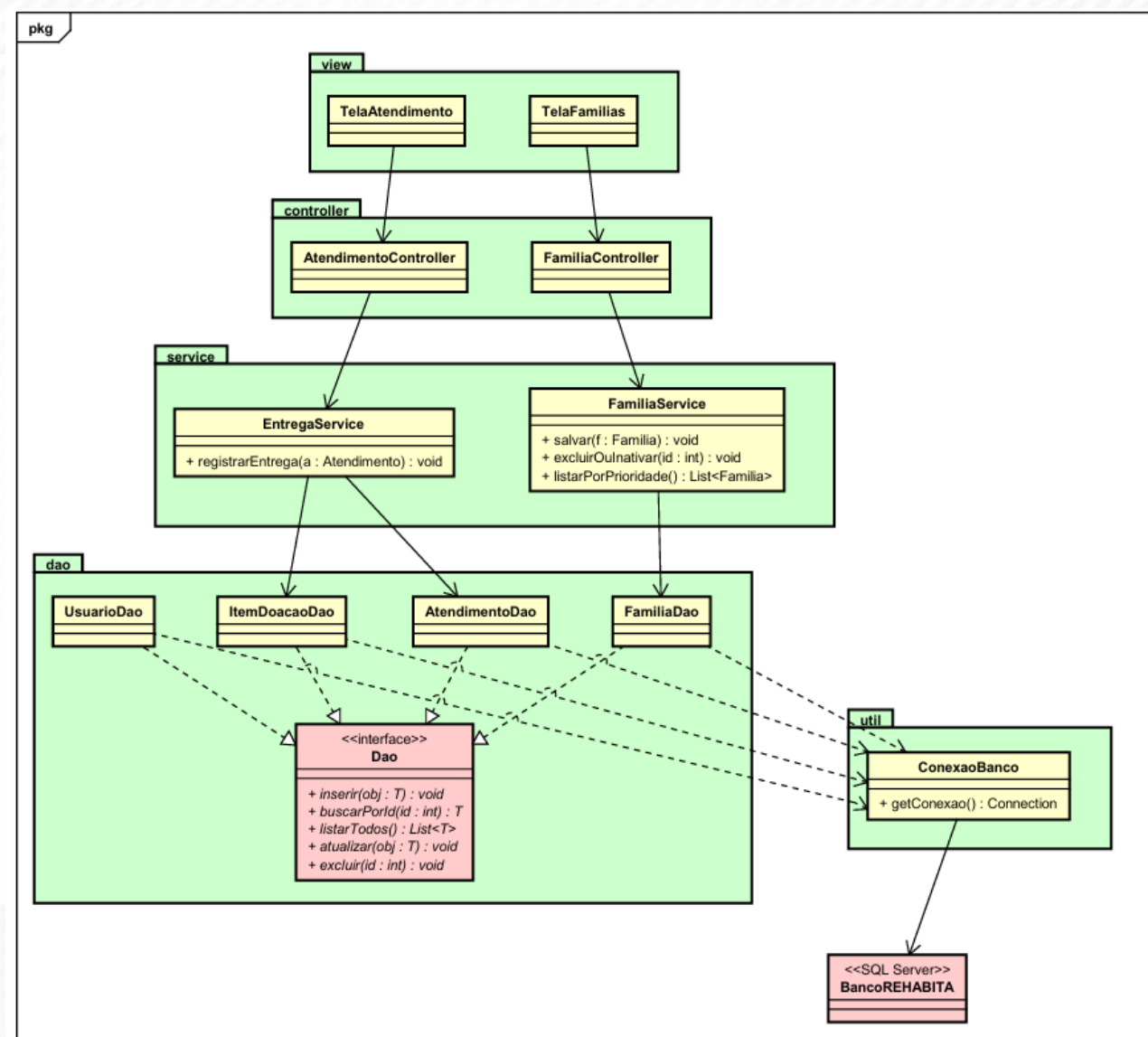
Salva tudo ou não salva nada. Ao registrar uma entrega, o sistema grava a entrega e baixa o estoque. Se der problema no meio, o banco desfaz as duas coisas (RN3). Assim nunca sobra entrega registrada sem a baixa no estoque.

Vamos usar a versão Express, que é gratuita, instalada no próprio computador e aberta na porta 1433. O sistema não entra com o usuário sa: criamos um usuário só para ele, que pode ler e gravar dados, mas não pode criar nem apagar tabela. Como o banco guarda CPF, endereço e renda das famílias, que são dados protegidos pela LGPD, faz sentido limitar esse acesso desde o começo. Para mexer no banco usamos o SQL Server Management Studio (SSMS) no Windows ou a extensão SQL Server (mssql) do VS Code.

5.4 Arquitetura: camadas com MVC e DAO

O código fica dividido em camadas, e cada uma tem uma função só. A página não sabe nada de banco de dados, e a parte que conversa com o banco não sabe nada das regras. A figura 1 mostra como elas se comunicam.

Figura 1 - Organização do código em camadas



Fonte: elaborado pelo grupo (2026).

Quadro 5 – Pacotes do projeto

Pacote	O que faz	Exemplos
model	Guarda as classes do sistema (a parte de POO da seção 6).	Familia, Membro, ItemEntregue
controller	Recebe o que vem da página e devolve a resposta.	FamiliaController, AtendimentoController
service	Onde ficam as regras de negócio (RN1 a RN7).	FamiliaService, EntregaService
dao	A única parte que conversa com o banco de dados.	FamiliaDao, ItemDoacaoDao, UsuarioDao
util	Coisas usadas por várias camadas.	ValidadorCpf, SenhaUtil
templates	As páginas web e o menu de terminal usado nos testes.	familias.html, MenuTeste

Um exemplo de como funciona na prática. Quando a assistente social registra uma entrega (R6), a página avisa o controller, que chama a service. É na service que o sistema confere se tem saldo (RN3) e se a família já recebeu aquele item há pouco tempo (RN4). Só depois disso ela manda os DAOs gravarem no banco.

A vantagem de separar assim é poder mexer numa parte sem quebrar as outras. Se a regra dos 30 dias mudar, mexemos só na service; a página e o banco continuam iguais.

Essa separação também organiza o CRUD da família, que é a entidade principal. Todo DAO implementa a interface `Dao<T>`, escrevendo as cinco operações com `@Override`: inserir, buscar por id, listar todos, atualizar e excluir. Nas buscas, os dados digitados pelo usuário são enviados ao banco separados do comando, e não misturados dentro dele. Isso protege contra SQL injection, que é quando alguém escreve um pedaço de comando dentro de um campo da tela para tentar apagar ou roubar dados.

5.5 Telas

Como somos do curso de Engenharia de Software, o foco da nossa entrega é a arquitetura e o funcionamento do back-end, não o visual. Por isso cada parte do sistema é testada primeiro por um menu de terminal e só depois ligada nas páginas web: assim, se a interface der problema, as regras e o banco continuam funcionando e podem ser mostrados do mesmo jeito. Mesmo assim, como o sistema é web, as páginas precisam ser fáceis de usar, porque boa parte dos voluntários não tem muita prática com computador. Por isso decidimos: mostrar as listas em tabela, com os filtros de bairro, status e urgência logo no topo; usar caixas de seleção nos campos de valor fixo, para ninguém digitar errado; marcar os campos obrigatórios e escrever mensagens de erro claras, como "Saldo insuficiente: restam 3 cestas básicas"; e montar o menu de acordo com o perfil, para a assistente social nem ver as opções que são só do coordenador (R8).

As páginas previstas são: login, menu principal, famílias (lista e cadastro, com membros e histórico), atendimento com entregas, doações recebidas, itens e estoque, usuários e consultas.

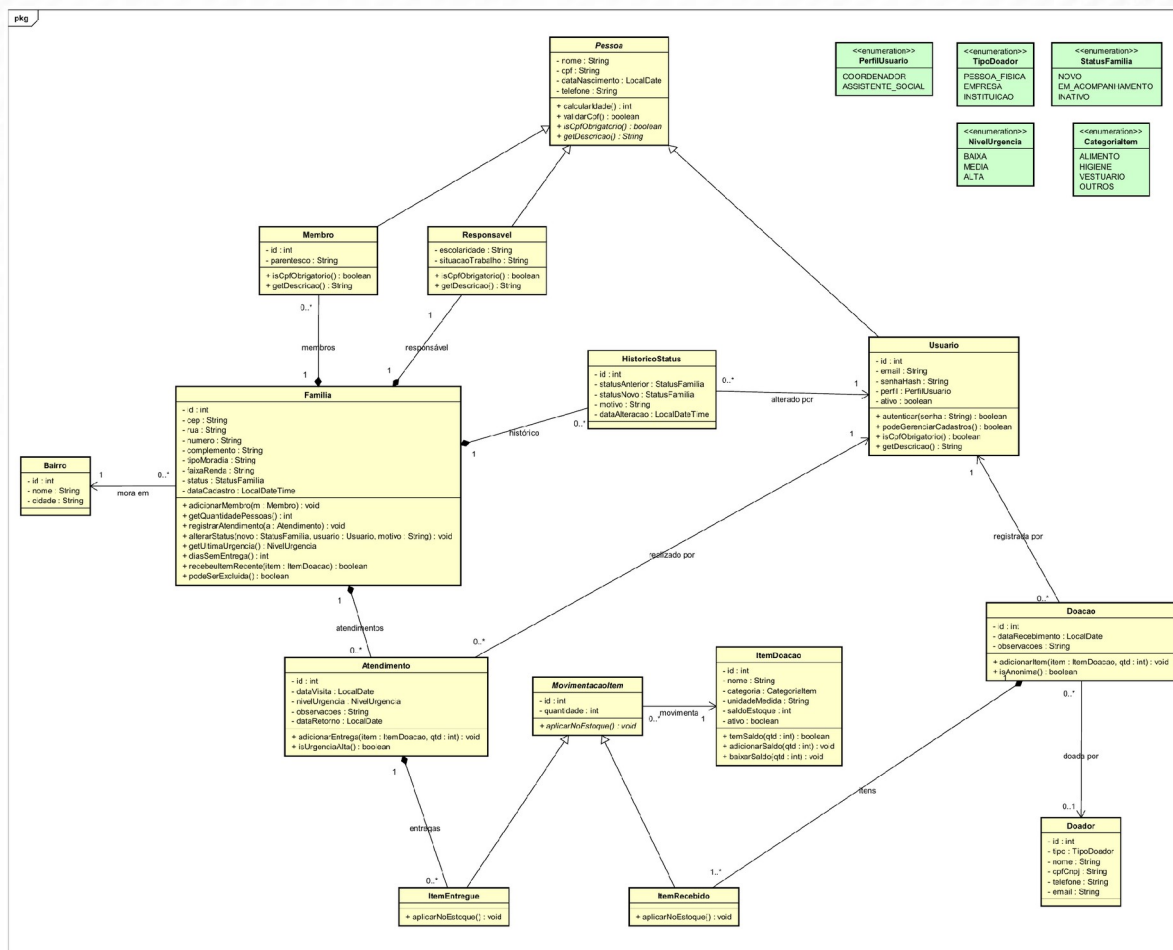
5.6 Ferramentas de apoio

Git e GitHub para guardar as versões do código; Maven para organizar o projeto; IntelliJ IDEA Community; SSMS ou VS Code para mexer no banco; Astah UML para o diagrama de classes. Obs: o Diagrama DER foi tirado dentro do SSMS na Ferramenta de Diagramas.

6. Diagrama de classes

A figura 2 mostra as classes do pacote model. A classe central é Família, que reúne o responsável, os membros, o histórico de status e os atendimentos. A parte de estoque gira em torno de ItemDoacao, com as doações recebidas (Doacao) de um lado e as entregas feitas nos atendimentos do outro. Para o diagrama não ficar poluído, os atributos que são outras classes (por

Figura 2 – Diagrama de classes do REHABITA



Fonte: elaborado pelo grupo (2026).

Polimorfismo. Pessoa tem dois métodos abstratos que cada subclasse sobrescreve usando `@Override`, que é a marcação do Java para dizer "este método está reescrevendo o da classe mãe". O `isCpfObrigatorio()` é usado pelo `validarCpf()`: `Responsavel` e `Usuario` retornam `true`, e `Membro` retorna `false`, porque criança muitas vezes ainda não tem CPF. O `getDescricao()` monta o texto que aparece nas telas, por exemplo "Maria Souza (responsável, 42 anos)" ou "Pedro Souza (filho, 8 anos)". Em `MovimentacaoItem` está o caso mais importante, o `aplicarNoEstoque()`: em `ItemRecebido` ele soma a quantidade no saldo do item e em `ItemEntregue` ele confere se há saldo e desconta. A `service` percorre uma lista de movimentações e chama `aplicarNoEstoque()` sem precisar saber de qual tipo é cada uma. Além das classes abstratas, a interface `Dao<T>` da figura 1 também é implementada com `@Override` por todos os DAOs.

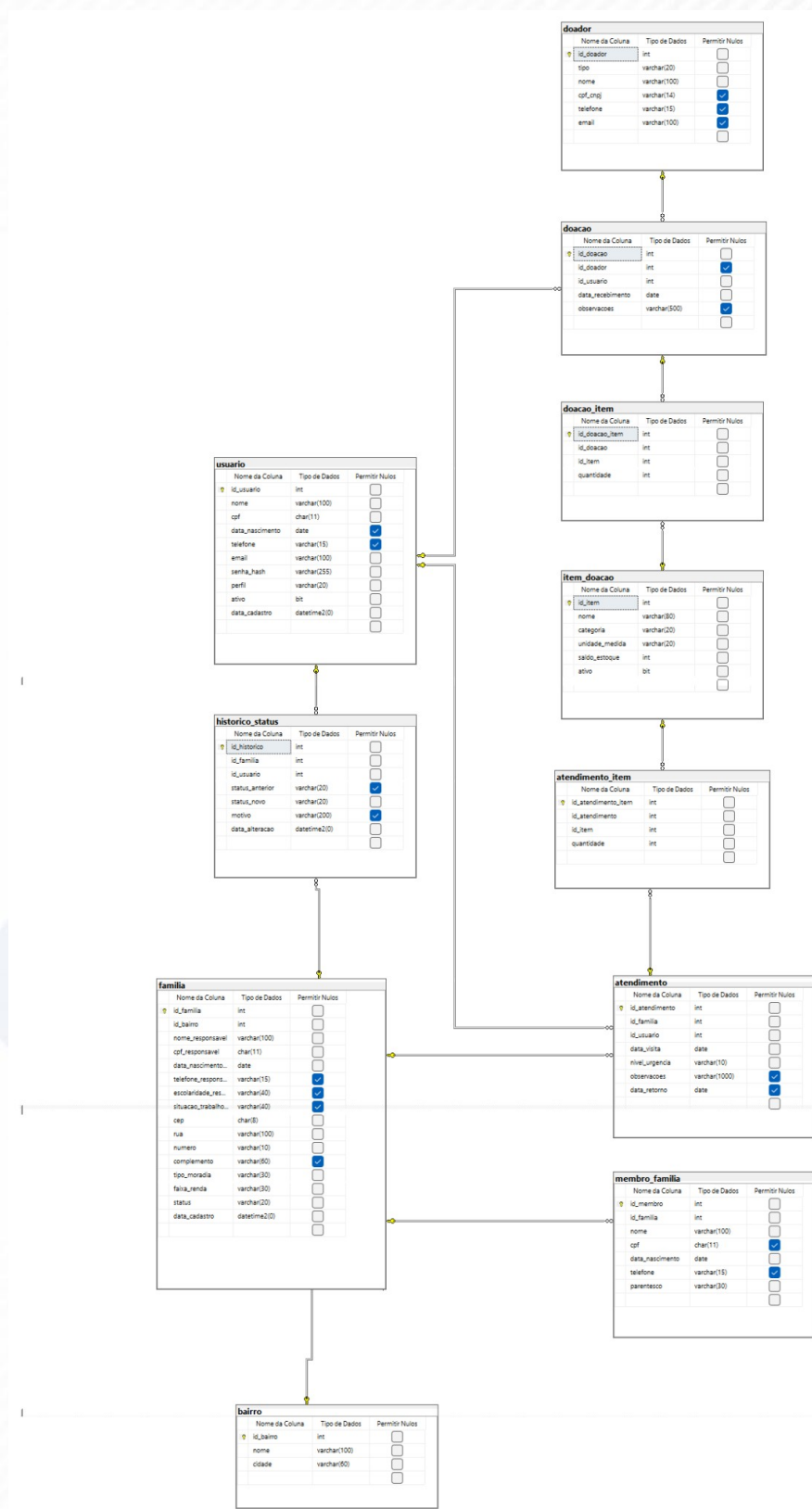
Composição 1:N. Família é composta por Membro (0..*), HistoricoStatus (0..*) e Atendimento (0..*); Atendimento é composto por ItemEntregue (0..*); e Doacao é composta por ItemRecebido (1..*, porque não existe doação sem item). Usamos composição porque essas partes não existem sozinhas: um membro não faz sentido sem a família e um item entregue não existe fora do atendimento. Responsavel também é uma composição de Família, mas com multiplicidade 1:1.

Associações. Família aponta para Bairro; Atendimento, HistoricoStatus e Doacao apontam para o Usuario que fez o registro; Doacao aponta para Doador com multiplicidade 0..1, porque a doação pode ser anônima; e toda MovimentacaoItem aponta para um ItemDoacao.

Enums. StatusFamília, NivelUrgencia, PerfilUsuario, CategoricalItem e TipoDoador guardam os valores fixos do sistema.

7. Diagrama do banco de dados (DER)

Figura 3 – Diagrama entidade-relacionamento



Quadro 6 - Correspondência entre classes e tabelas

Classe (Java)	Tabela (SQL Server)	Observação
Pessoa	-	Classe abstrata: não vira tabela.
Responsavel	familia	Fica dentro da tabela da família.
Membro	membro_familia	
Usuario	usuario	
Familia	familia	
Bairro	bairro	
HistoricoStatus	historico_status	
Atendimento	atendimento	
ItemDoacao	item_doacao	
MovimentacaoItem	-	Classe abstrata: não vira tabela.
ItemRecebido	doacao_item	Entrada no estoque.
ItemEntregue	atendimento_item	Saída do estoque.
Doacao	doacao	
Doador	doador	

A quantidade de pessoas da família não tem coluna, ela é sempre calculada (RN2), para nunca ficar diferente do número de membros cadastrados. O mesmo vale para a urgência atual da família, que é sempre a do último atendimento.

8. Controle de versão e estrutura do repositório

O projeto está versionado no GitHub, em repositório público:

<https://github.com/JuliaYumii/Re-habita---AEP-4-Semestre>

Quadro 7 – Estrutura do repositório

Pasta / arquivo	Conteúdo
/src/main/java/br/com/rehabita/	Pacotes model, controller, service, dao e util (o código entra na 2ª entrega).
/src/main/resources/templates/	As páginas HTML do sistema.
/src/main/resources/	Arquivo de configuração com os dados de conexão do banco.
/database/	Scripts que criam o banco e as tabelas.
/docs/	Este documento em PDF.
/docs/diagramas/	Diagrama de classes, arquitetura em camadas e DER.
README.md	Descrição, ODS, requisitos, regras de negócio, cronograma e como rodar o sistema.
.gitignore	Arquivos que não vão para o GitHub, como o que guarda a senha do banco.

Referências

MICROSOFT. **Microsoft JDBC Driver for SQL Server**. Repositório oficial. Disponível em: <https://github.com/microsoft/mssql-jdbc>. Acesso em: 10 set. 2026.

NAÇÕES UNIDAS BRASIL. **Objetivos de Desenvolvimento Sustentável**. Disponível em: <https://brasil.un.org/pt-br/sdgs>. Acesso em: 10 set. 2026.

PLANTUML. **PlantUML: documentação**. Disponível em: <https://plantuml.com>. Acesso em: 10 set. 2026.

SPRING. **Spring Boot: documentação de referência**. Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: 10 set. 2026.

UNICESUMAR. **Guia da AEP: 4º semestre (2026.2)**. Maringá: Unicesumar, 2026. Material da disciplina.